

## **EJERCICIO N°2: SISTEMA BANCARIO JM DESARROLLADO EN JAVA**

### **1. RESUMEN**

El presente proyecto consiste en el desarrollo de un sistema bancario básico implementado en el lenguaje de programación Java utilizando Programación Orientada a Objetos (POO). El sistema permite la creación de cuentas bancarias, generación automática de números de cuenta, realización de depósitos y retiros, consulta de información de clientes, visualización del historial de movimientos y conteo total de cuentas registradas.

La aplicación fue desarrollada para ejecutarse en consola y emplea estructuras de datos dinámicas mediante ArrayList para almacenar las cuentas creadas. Asimismo, incorpora mecanismos de validación de datos y manejo de excepciones para garantizar la integridad de la información ingresada por los usuarios.

Palabras clave: Java, Programación Orientada a Objetos, Sistema Bancario, Gestión de Cuentas, Historial de Transacciones.

### **2. INTRODUCCIÓN**

Los sistemas bancarios constituyen una de las aplicaciones más importantes dentro del desarrollo de software debido a la necesidad de gestionar información financiera de manera segura y eficiente. En la actualidad, las entidades financieras utilizan sistemas informáticos para administrar cuentas, registrar transacciones y controlar saldos de clientes.

En este contexto, se desarrolló un sistema bancario básico utilizando Java con el propósito de aplicar los conceptos fundamentales de la Programación Orientada a Objetos, tales como encapsulamiento, clases, objetos, atributos y métodos. El sistema implementado permite simular operaciones bancarias comunes mediante una interfaz de consola sencilla e intuitiva.

### **3. OBJETIVOS**

- **Objetivo General:**

Desarrollar un sistema bancario básico en Java que permita administrar cuentas bancarias y realizar operaciones financieras elementales mediante Programación Orientada a Objetos.

- **Objetivos Específicos:**

- Implementar la creación de cuentas bancarias con generación automática de números de cuenta.
- Registrar depósitos y retiros de dinero.
- Validar los datos ingresados por los usuarios.
- Mantener un historial de movimientos por cuenta.
- Gestionar múltiples cuentas utilizando estructuras dinámicas.
- Aplicar manejo de excepciones para controlar errores de ejecución.

#### **4. MARCO TEÓRICO**

- Programación Orientada a Objetos

La Programación Orientada a Objetos es un paradigma de programación basado en la creación de objetos que contienen atributos y métodos. Este enfoque facilita la organización, reutilización y mantenimiento del código.

- Clase

Una clase es una plantilla que define las características y comportamientos de un objeto.

En el proyecto se implementan las clases:

- CuentaBancaria
- Principal

- Objeto

Un objeto es una instancia de una clase. Cada cuenta bancaria creada corresponde a un objeto de la clase CuentaBancaria.

- Encapsulamiento

Consiste en proteger los atributos de una clase mediante modificadores de acceso privados y permitir su acceso controlado mediante métodos públicos.

- ArrayList

ArrayList es una estructura de datos dinámica que permite almacenar elementos de manera flexible.

En el proyecto se utiliza para almacenar todas las cuentas bancarias registradas.

- Manejo de Excepciones

Las excepciones permiten detectar y controlar errores durante la ejecución del programa.

El sistema utiliza bloques try-catch para controlar errores relacionados con depósitos, retiros y entradas inválidas.

#### **5. ANÁLISIS DEL PROBLEMA**

- 1) Situación Problemática

La administración manual de cuentas bancarias puede generar errores en el registro de transacciones y dificultar el control de saldos de los clientes. Por

ello resulta necesario implementar una solución informática que permita gestionar la información de manera organizada y segura.

2) Requerimientos Funcionales:

- Crear cuentas bancarias.
- Generar números de cuenta automáticamente.
- Realizar depósitos.
- Realizar retiros.
- Mostrar información de cuentas.
- Visualizar historial de movimientos.
- Contabilizar el total de cuentas creadas.

3) Requerimientos No Funcionales:

- Facilidad de uso.
- Correcta validación de datos.
- Mantenimiento sencillo.
- Código estructurado mediante POO.

## 6. DISEÑO DE LA SOLUCIÓN

- Clase CuentaBancaria:  
Responsable de almacenar y gestionar la información de cada cliente.

- Atributos:

- Nombre
- Dni
- Correo
- NumeroCuenta
- Saldo
- Historial
- TotalCuentas
- Correlative

- Métodos:

- depositar()
- retirar()
- registrarMovimiento()
- mostrarDatos()
- getNombre()
- getDni()
- getCorreo()
- getNumeroCuenta()
- getSaldo()

- getHistorial()
- getTotalCuentas()

- Clase Principal:

Responsable de controlar el flujo general del programa.

- Métodos:
  - validarDni()
  - buscarCuenta()
  - main()

## 7. ANÁLISIS DEL FUNCIONAMIENTO

El sistema inicia cargando seis cuentas bancarias predefinidas.

Posteriormente se muestra un menú con siete opciones:

- 1) Crear cuenta.
- 2) Depositar dinero.
- 3) Retirar dinero.
- 4) Mostrar cuentas.
- 5) Ver historial.
- 6) Mostrar total de cuentas.
- 7) Salir.

Cada operación es procesada mediante estructuras de control switch y validaciones específicas.

Para evitar errores:

- El DNI debe contener exactamente ocho dígitos.
- Los depósitos deben ser mayores que cero.
- Los retiros deben ser mayores que cero.
- No se permite retirar montos superiores al saldo disponible.
- Además, cada movimiento realizado se almacena automáticamente en el historial de la cuenta correspondiente.

## 8. CASOS DE PRUEBA:

- ✓ Caso 1: Creación de Cuenta

Entrada:

Nombre: Pedro Sánchez

DNI: 44556677

Saldo Inicial: S/1000

Resultado Esperado:

Cuenta creada correctamente.

Generación automática de número de cuenta.

Resultado Obtenido:

Correcto.

✓ Caso 2: Depósito

Entrada:

DNI: 12345678

Monto: S/500

Resultado Esperado:

Incremento del saldo.

Resultado Obtenido:

Correcto.

✓ Caso 3: Retiro

Entrada:

DNI: 12345678

Monto: S/200

Resultado Esperado:

Disminución del saldo.

Resultado Obtenido:

Correcto.

✓ Caso 4: Retiro Superior al Saldo

Entrada:

Monto mayor al saldo disponible.

Resultado Esperado:

Mensaje de saldo insuficiente.

Resultado Obtenido:

Correcto.

## **9. RESULTADOS**

Durante las pruebas realizadas se verificó que el sistema permite gestionar múltiples cuentas bancarias de forma eficiente.

La generación automática de números de cuenta evita duplicidades y facilita la identificación de clientes. Asimismo, el historial de movimientos permite mantener un registro detallado de todas las operaciones efectuadas.

Las validaciones implementadas garantizan la integridad de los datos y reducen la posibilidad de errores operativos.

## **10. CONCLUSIONES**

- 1) La Programación Orientada a Objetos permitió estructurar adecuadamente el sistema mediante clases y objetos.
- 2) El uso de ArrayList facilitó la gestión dinámica de múltiples cuentas bancarias.
- 3) Las validaciones implementadas mejoraron la confiabilidad del sistema.
- 4) El manejo de excepciones permitió controlar errores sin interrumpir la ejecución del programa.
- 5) El sistema cumple satisfactoriamente con los requisitos planteados para la administración básica de cuentas bancarias.

## **11. RECOMENDACIONES**

- 1) Incorporar persistencia de datos mediante archivos o bases de datos.
- 2) Implementar autenticación de usuarios.
- 3) Desarrollar una interfaz gráfica utilizando JavaFX o Swing.
- 4) Agregar operaciones de transferencia entre cuentas.
- 5) Implementar cifrado de información sensible para incrementar la seguridad.

## **12. BIBLIOGRAFÍA**

Deitel, P. y Deitel, H. Java: How to Program. Pearson Education.

Sierra, K. y Bates, B. Head First Java. O'Reilly Media.

Oracle Corporation. Documentación Oficial de Java.

Eckel, B. Thinking in Java. Prentice Hall.